

Bibliothèques

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

# Optionnel : affichage plus joli
sns.set(style="whitegrid")
```

1. Objectif du TP

- Charger le dataset **Iris**.
- Explorer sa structure (lignes, colonnes, types, valeurs manquantes).
- Visualiser les distributions des variables.
- Calculer les **statistiques descriptives** (moyenne, médiane, quartiles, variance, écart-type).
- Détecter les **valeurs aberrantes (outliers)** avec la méthode de l'**IQR**.

2. Code complet du TP

1) Chargement du dataset Iris

```
df = sns.load_dataset("iris") # Dataset Iris (150 lignes, 5 colonnes)
```

2) Inspection de base

2.1. Aperçu des premières lignes

```
print("=== Head (5 premières lignes) ===")
print(df.head(), "\n")
```

2.2. Aperçu des dernières lignes

```
print("=== Tail (5 dernières lignes) ===")
print(df.tail(), "\n")
```

2.3. Dimensions du DataFrame

```
print("=== Shape (nb_lignes, nb_colonnes) ===")
print(df.shape, "\n")
```

2.4. Types des colonnes

```
print("=== Types des colonnes ===")
print(df.dtypes, "\n")
```

2.5. Valeurs manquantes par colonne

```
print("=== Valeurs manquantes par colonne ===")
print(df.isna().sum(), "\n")
```

2.6. Nombre total de valeurs manquantes

```
print("=== Nombre total de valeurs manquantes ===")
print(df.isna().sum().sum(), "\n")
```

2.7. Informations générales

```
print("=== Info ===")
print(df.info(), "\n")
```

2.8. Noms des colonnes

```
print("=== Noms des colonnes ===")
print(df.columns, "\n")
```

3) Visualisation des distributions (histogrammes)

```
# On ne garde que les colonnes numériques
numeric_cols = df.select_dtypes(include=["float", "int"]).columns

for col in numeric_cols:
    plt.figure(figsize=(8, 4))
    sns.histplot(df[col], kde=True)
    plt.title(f"Histogramme de {col}")
    plt.xlabel(col)
    plt.ylabel("Fréquence")
    plt.show()
```

4) Résumé statistique automatique

```
print("=== Description statistique (describe) ===")
print(df[numeric_cols].describe(), "\n")
```

5) Fonctions de statistiques descriptives

```
def calculer_moyenne(df, colonne):
    return df[colonne].mean()

def calculer_mediane(df, colonne):
    return df[colonne].median()

def calculer_quartiles(df, colonne):
    Q1 = df[colonne].quantile(0.25)
    Q2 = df[colonne].median()
    Q3 = df[colonne].quantile(0.75)
    return Q1, Q2, Q3

def calculer_variance_et_ecart_type(df, colonne):
    variance = df[colonne].var()
    ecart_type = df[colonne].std()
    return variance, ecart_type

def calculer_etendue_interquartile(Q1, Q3):
```

```

return Q3 - Q1

print("=== Statistiques descriptives détaillées par variable ===")
for col in numeric_cols:
    print(f"\n--- {col} ---")
    moy = calculer_moyenne(df, col)
    med = calculer_mediane(df, col)
    Q1, Q2, Q3 = calculer_quartiles(df, col)
    var, std = calculer_variance_et_ecart_type(df, col)
    IQR = calculer_etendue_interquartile(Q1, Q3)

    print(f"Moyenne           : {moy:.2f}")
    print(f"Médiane           : {med:.2f}")
    print(f"Quartile 1 (Q1)       : {Q1:.2f}")
    print(f"Quartile 2 (Q2)       : {Q2:.2f}")
    print(f"Quartile 3 (Q3)       : {Q3:.2f}")
    print(f"Variance              : {var:.2f}")
    print(f"Écart-type            : {std:.2f}")
    print(f"IQR (Q3 - Q1)         : {IQR:.2f}")

```

6) Détection des valeurs aberrantes (IQR)

```

print("\n=== Détection des outliers (méthode IQR) ===")
for col in numeric_cols:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1

    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    # masque booléen : True = valeur aberrante
    outliers_mask = (df[col] < lower_bound) | (df[col] > upper_bound)
    nb_outliers = outliers_mask.sum()

    print(f"{col} : {nb_outliers} valeurs aberrantes")

    # (optionnel) afficher les lignes concernées
    # print(df.loc[outliers_mask, [col, "species"]])

```

7) Visualisation des outliers (boxplots)

```

for col in numeric_cols:
    plt.figure(figsize=(6, 4))
    sns.boxplot(x=df[col])
    plt.title(f"Boxplot de {col}")
    plt.xlabel(col)
    plt.show()

```

8) (Optionnel) Chargement Iris via scikit-learn

```
from sklearn.datasets import load_iris

iris_skl = load_iris()
df_skl = pd.DataFrame(data=iris_skl.data,
                      columns=iris_skl.feature_names)
df_skl["species"] = iris_skl.target

print("=== Aperçu du DataFrame Iris (scikit-learn) ===")
print(df_skl.head())
```

3. Étapes

1. Chargement des données

- `sns.load_dataset("iris")` pour créer `df`.

2. Inspection

- `head(), tail(), shape(), dtypes(), isna().sum(), info(), describe()`.

3. Visualisation des distributions

- Boucle sur les colonnes numériques → `sns.histplot()` pour chaque variable.

4. Statistiques descriptives

- Fonctions pour calculer moyenne, médiane, Q1, Q3, variance, écart-type, IQR.
- Boucle pour afficher ces statistiques par variable.

5. Détection des outliers

- Méthode IQR :
 - `Q1, Q3, IQR = Q3 - Q1,`
 - `lower_bound = Q1 - 1.5*IQR, upper_bound = Q3 + 1.5*IQR,`
 - masque booléen pour compter les valeurs hors bornes.

6. Visualisation des outliers

- `sns.boxplot()` pour chaque variable numérique.

7. Optionnel

- Chargement du même dataset via `sklearn.datasets.load_iris` dans un autre DataFrame.